

Instruções:

1. Esta atividade deve ser realizada em **Duplas** ou **Trios**.
2. A defesa da implementação e a data final da postagem do código fonte comentado estarão devidamente indicadas no AVA2. Defesas/entregas feitas na aula seguinte terão 70% da nota.
3. A implementação poderá ser realizada em **qualquer** linguagem de programação, sendo que o código deverá ser entregue (também é válido o link para o trabalho no GitHub) e apresentado.
4. Todos os integrantes do grupo deverão estar presentes para apresentação e os códigos deverão conter identificação dos autores. Se algum integrante faltar, este terá que apresentar sozinho na semana seguinte e estará sujeito a *instrução 2*.
5. Se forem identificados trabalhos com grau de similaridade que caracterize cópia ou adaptação (autorizadas ou não pelos seus autores originais), a nota dos grupos será a nota de um trabalho dividida pelo número de grupos que entregou esses trabalhos similares.

1. Objetivo

Implementar um simulador de memória cache capaz de reproduzir o comportamento de políticas de mapeamento direto e por conjunto (set-associative). O programa deverá ler sequências de endereços de memória a partir de um arquivo, simular os acessos na cache configurada e exibir estatísticas detalhadas de desempenho.

2. Contextualização

A memória cache é um componente essencial na hierarquia de memória de processadores modernos. Seu desempenho depende diretamente de como os endereços de memória são decompostos em campos de tag, index e offset, além da política de mapeamento utilizada.

Neste trabalho, o aluno deverá compreender e implementar os dois esquemas mais comuns:

- **Mapeamento direto** — cada bloco da memória principal pode ocupar apenas uma linha específica da cache.
- **Mapeamento por conjunto** — a cache é dividida em conjuntos; cada bloco pode ser alocado em qualquer linha dentro de seu conjunto.

3. Especificação do Programa

3.1 Parâmetros de Entrada

O programa deve receber, via linha de comando ou menu interativo, os seguintes parâmetros:

Parâmetro	Descrição	Exemplo
--cache-size	Tamanho total da cache (em bytes)	1024
--block-size	Tamanho de cada bloco/linha (em bytes)	16
--assoc	Grau de associatividade (1 = direta; N = N-way)	1, 2, 4
--addr-bits	Número de bits do endereço físico	16
--input	Caminho do arquivo com endereços	acessos.txt

A partir dos parâmetros fornecidos, o programa deve calcular e exibir automaticamente a decomposição dos endereços: número de bits de offset, index e tag, antes de iniciar a simulação.

3.2 Formato do Arquivo de Endereços

O arquivo de entrada deve conter um endereço por linha, podendo estar em formato decimal ou hexadecimal (prefixo 0x). Linhas iniciadas com # devem ser ignoradas (comentários). Exemplo:

```
# Exemplo de arquivo de endereços
0x0010
0x0024
0x0010
256
0x00FF
```

3.3 Política de Substituição

Para o mapeamento por conjunto (associatividade > 1), as políticas abaixo:

- **LRU** (Least Recently Used)
- **FIFO** (First In, First Out)

4. Saída Esperada

Ao final da simulação, o programa deve imprimir um relatório contendo, no mínimo:

- Configuração utilizada (tamanho da cache, bloco, associatividade, bits de tag/index/offset).
- Total de acessos realizados.
- Número e percentual de cache hits.
- Número e percentual de cache misses.

Adicionalmente, com a flag --verbose, o programa deve exibir o estado da cache linha a linha após cada acesso, indicando o endereço acessado, os campos decompostos (tag, index, offset) e se ocorreu hit ou miss.

5. Exemplos de Configuração

A tabela a seguir apresenta exemplos de configurações válidas e os respectivos campos de endereço esperados. Utilize-os como referência para validar a sua implementação:

Cache (bytes)	Bloco (bytes)	Assoc.	Endereço (bits)	Offset (bits)	Index (bits)	Tag (bits)
256	16	1 (direta)	16	4	4	8
1024	32	2-way	16	5	4	7
512	8	4-way	16	3	4	9

2048	64	8-way	32	6	2	24
------	----	-------	----	---	---	----

6. Requisitos Técnicos

- Tratamento obrigatório de entradas inválidas: tamanho de cache que não seja potência de 2, arquivo inexistente, endereço fora do espaço de endereçamento, parâmetros inconsistentes, entre outros.
- O código deve estar organizado em módulos e funções bem nomeados e comentados.
- Deve ser entregue um arquivo README.md explicando como compilar, executar e apresentando exemplos de uso.
- Ao menos dois arquivos de teste (*.txt) com sequências de endereços devem acompanhar a entrega.

7. Entrega e Avaliação

7.1 Forma de Entrega

A entrega deve ser realizada via repositório Git (GitHub ou GitLab) ou arquivo .zip conforme orientação do professor, contendo:

1. Código-fonte completo e organizado.
2. Arquivo README.md com instruções de compilação e execução.
3. Ao menos dois arquivos de teste com endereços de memória.

7.2 Critérios de Avaliação

Critério	Pontos
Corretude da simulação — mapeamento direto	4 pts
Corretude da simulação — mapeamento por conjunto (N-way + LRU)	3 pts
Leitura correta do arquivo e tratamento de erros	1 pts
Qualidade do código, modularização e comentários	1 pts
README e documentação	1 pts

Atenção

Trabalhos com evidência de cópia ou uso indevido de código gerado por IA sem declaração e justificativa serão anulados. Em caso de dúvidas sobre a autoria, o aluno poderá ser convocado para defesa oral do trabalho.